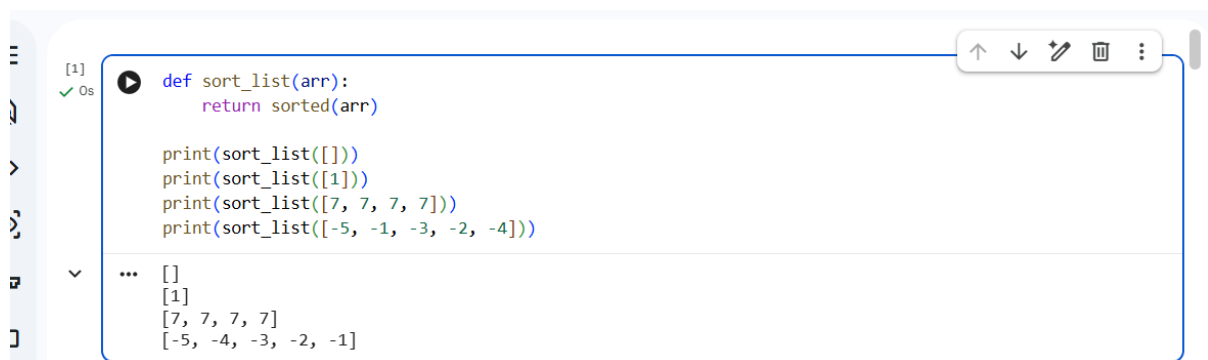


TOPIC 2 : BRUTE FORCE

1. Write a program to perform the following

- o An empty list
- o A list with one element
- o A list with all identical elements
- o A list with negative numbers



```
[1]
✓ 0s ▶ def sort_list(arr):
      return sorted(arr)

      print(sort_list([]))
      print(sort_list([1]))
      print(sort_list([7, 7, 7, 7]))
      print(sort_list([-5, -1, -3, -2, -4]))

  ...  []
      [1]
      [7, 7, 7, 7]
      [-5, -4, -3, -2, -1]
```

2. Describe the Selection Sort algorithm's process of sorting an array. Selection Sort works by dividing the array into a sorted and an unsorted region. Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm.

```
[2]
✓ Os
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

print(selection_sort([5, 2, 9, 1, 5, 6]))
print(selection_sort([10, 8, 6, 4, 2]))
print(selection_sort([1, 2, 3, 4, 5]))

... [1, 2, 5, 5, 6, 9]
     [2, 4, 6, 8, 10]
     [1, 2, 3, 4, 5]
```

3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

```
[3]
✓ Os
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr

print(bubble_sort([64, 25, 12, 22, 11]))
print(bubble_sort([29, 10, 14, 37, 13]))
print(bubble_sort([3, 5, 2, 1, 4]))
print(bubble_sort([1, 2, 3, 4, 5]))
print(bubble_sort([5, 4, 3, 2, 1]))

... [11, 12, 22, 25, 64]
     [10, 13, 14, 29, 37]
     [1, 2, 3, 4, 5]
     [1, 2, 3, 4, 5]
     [1, 2, 3, 4, 5]
```

4. Test Cases:

☐ Test your optimized function with the following lists:

1. Input: [64, 25, 12, 22, 11]

□ Expected Output: [11, 12, 22, 25, 64]

2. Input: [29, 10, 14, 37, 13]

□ Expected Output: [10, 13, 14, 29, 37]

3. Input: [3, 5, 2, 1, 4]

□ Expected Output: [1, 2, 3, 4, 5]

4. Input: [1, 2, 3, 4, 5] (Already sorted list)

□ Expected Output: [1, 2, 3, 4, 5]

5. Input: [5, 4, 3, 2, 1] (Reverse sorted list)

□ Expected Output: [1, 2, 3, 4, 5]

1. Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome.

```
[4]
✓ 0s
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr

print(insertion_sort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3]))
print(insertion_sort([5, 5, 5, 5, 5]))
print(insertion_sort([2, 3, 1, 3, 2, 1, 1, 3]))

... [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
     [5, 5, 5, 5, 5]
     [1, 1, 1, 2, 2, 3, 3, 3]
```

5. Given an array arr of positive integers sorted in a strictly increasing order, and an integer k. return the kth positive integer that is missing from this array.

```

5]
✓ 0s
def find_kth_missing(arr, k):
    missing = 0
    current = 1

    while True:
        if current not in arr:
            missing += 1

            if missing == k:
                return current

        current += 1

print(find_kth_missing([2, 3, 4, 7, 11], 5))
print(find_kth_missing([1, 2, 3, 4], 2))
... 9
6

```

6. A peak element is an element that is strictly greater than its neighbors. Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in $O(\log n)$ time.

```

6]
✓ 0s
def find_peak(nums):
    left = 0
    right = len(nums) - 1

    while left < right:
        mid = (left + right) // 2

        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid

    return left

print(find_peak([1, 2, 3, 1]))
print(find_peak([1, 2, 1, 3, 5, 6, 4]))
... 2
5

```

7. Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

```
[7]
✓ 0s
def str_str(haystack, needle):
    n = len(haystack)
    m = len(needle)

    if m == 0:
        return 0

    for i in range(n - m + 1):
        if haystack[i:i + m] == needle:
            return i

    return -1

print(str_str("sadbutsad", "sad"))
print(str_str("leetcode", "leeto"))
```

... 0
-1

8. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

```
[8]
✓ 0s
def string_matching(words):
    result = []

    for i in range(len(words)):
        for j in range(len(words)):
            if i != j and words[i] in words[j]:
                result.append(words[i])
                break

    return result

print(string_matching(["mass", "as", "hero", "superhero"]))
print(string_matching(["leetcode", "et", "code"]))
print(string_matching(["blue", "green", "bu"]))
```

... ['as', 'hero']
['et', 'code']
[]

9. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach.

```

[9] 0s
def distance(p1, p2):
    return math.sqrt(
        (p1[0] - p2[0]) ** 2 +
        (p1[1] - p2[1]) ** 2
    )

def closest_pair(points):
    min_distance = float('inf')
    closest = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = distance(points[i], points[j])

            if d < min_distance:
                min_distance = d
                closest = (points[i], points[j])

    return closest, min_distance

points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, d = closest_pair(points)

print("Closest pair:", pair)
print("Minimum distance:", d)

... Closest pair: ((1, 2), (3, 1))
    Minimum distance: 2.23606797749979

```

10. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline? Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5). output: P3, P4, P6, P5, P7, P1

Code:

def orientation(a, b, c):

```
value = ((b[0] - a[0]) * (c[1] - a[1]) -  
         (b[1] - a[1]) * (c[0] - a[0]))
```

```
if value > 0:
```

```
    return 1
```

```
elif value < 0:
```

```
    return -1
```

```
return 0
```

```
def convex_hull(points):
```

```
    hull = []
```

```
    n = len(points)
```

```
    for i in range(n):
```

```
        for j in range(i + 1, n):
```

```
            left = False
```

```
            right = False
```

```
            for k in range(n):
```

```
                if k == i or k == j:
```

```
        continue
```

```
    o = orientation(points[i], points[j], points[k])
```

```
    if o > 0:
```

```
        left = True
```

```
    elif o < 0:
```

```
        right = True
```

```
    if not (left and right):
```

```
        if points[i] not in hull:
```

```
            hull.append(points[i])
```

```
        if points[j] not in hull:
```

```
            hull.append(points[j])
```

```
# Sort points around their center
```

```
cx = sum(p[0] for p in hull) / len(hull)
```

```
cy = sum(p[1] for p in hull) / len(hull)
```

```
import math
```

```
hull.sort(key=lambda p: math.atan2(p[1] - cy, p[0] - cx))
```

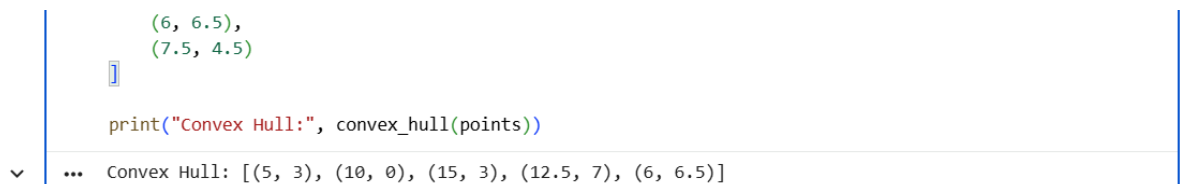


```
return hull
```

```
points = [  
    (10, 0),  
    (11, 5),  
    (5, 3),  
    (9, 3.5),  
    (15, 3),  
    (12.5, 7),  
    (6, 6.5),  
    (7.5, 4.5)  
]
```

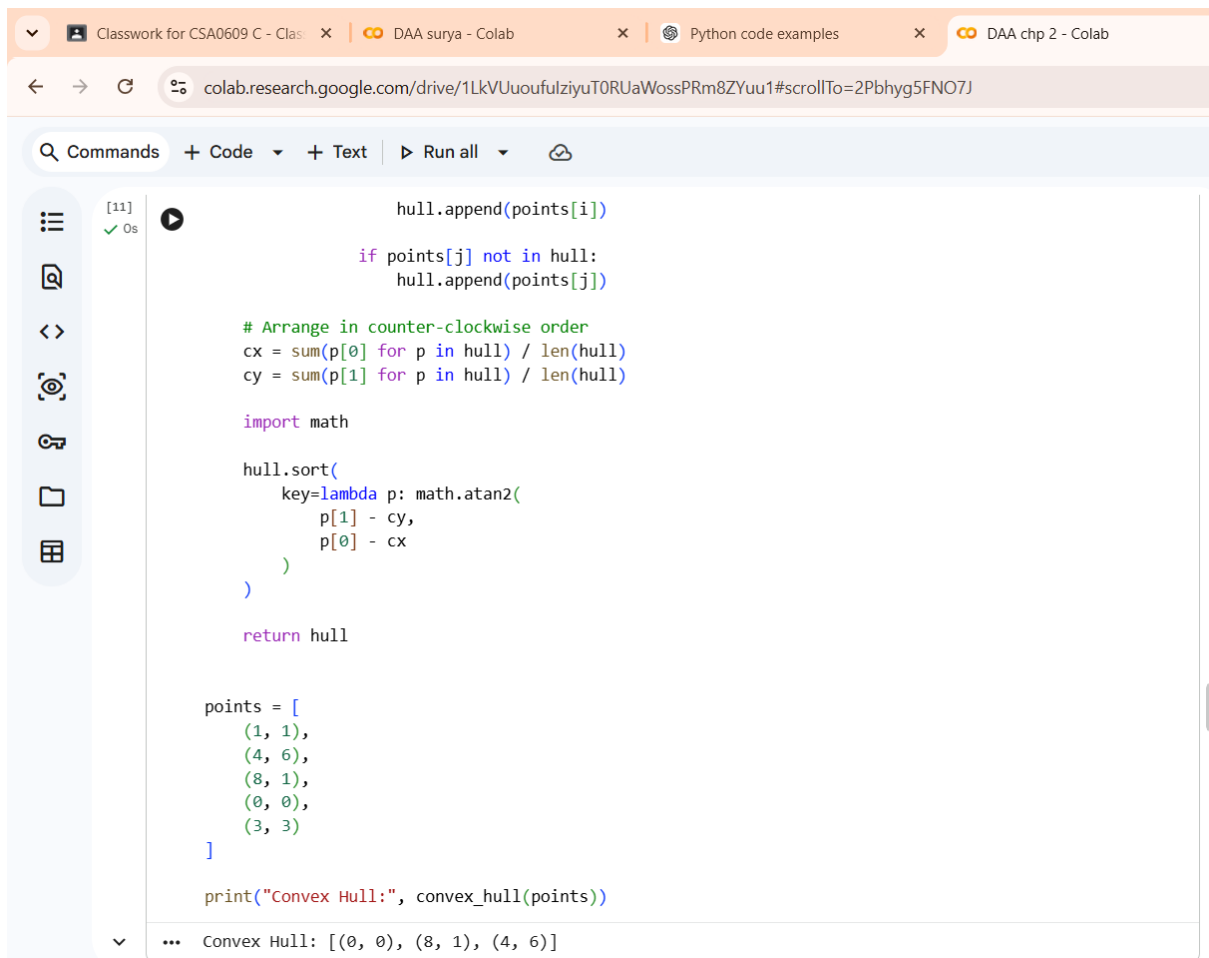
```
print("Convex Hull:", convex_hull(points))
```

Output:



```
(6, 6.5),  
(7.5, 4.5)  
]  
  
print("Convex Hull:", convex_hull(points))  
... Convex Hull: [(5, 3), (10, 0), (15, 3), (12.5, 7), (6, 6.5)]
```

11. Write a program that finds the convex hull of a set of 2D points using the brute force approach.



```
hull.append(points[i])

if points[j] not in hull:
    hull.append(points[j])

# Arrange in counter-clockwise order
cx = sum(p[0] for p in hull) / len(hull)
cy = sum(p[1] for p in hull) / len(hull)

import math

hull.sort(
    key=lambda p: math.atan2(
        p[1] - cy,
        p[0] - cx
    )
)

return hull

points = [
    (1, 1),
    (4, 6),
    (8, 1),
    (0, 0),
    (3, 3)
]

print("Convex Hull:", convex_hull(points))
```

... Convex Hull: [(0, 0), (8, 1), (4, 6)]

12. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

The screenshot shows a Google Colab notebook with the following content:

```
total = 0

for i in range(len(path) - 1):
    total += distance(path[i], path[i + 1])

if total < min_distance:
    min_distance = total
    best_path = path

return min_distance, best_path

cities1 = [(1, 2), (4, 5), (7, 1), (3, 6)]
cities2 = [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)]

distance1, path1 = tsp(cities1)
distance2, path2 = tsp(cities2)

print("Test Case 1")
print("Shortest Distance:", distance1)
print("Shortest Path:", path1)

print("\nTest Case 2")
print("Shortest Distance:", distance2)
print("Shortest Path:", path2)
```

The output of the code is as follows:

```
... Test Case 1
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

Test Case 2
Shortest Distance: 23.12995011084934
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]
```

13. You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

```
Q Commands + Code + Text ▶ Run all ☁

[13]
✓ Os ▶ return result, min_cost

cost1 = [
    [3, 10, 7],
    [8, 5, 12],
    [4, 6, 9]
]

cost2 = [
    [15, 9, 4],
    [8, 7, 18],
    [6, 12, 11]
]

assignment1, total1 = assignment_problem(cost1)
assignment2, total2 = assignment_problem(cost2)

print("Test Case 1")
print("Optimal Assignment:", assignment1)
print("Total Cost:", total1)

print("\nTest Case 2")
print("Optimal Assignment:", assignment2)
print("Total Cost:", total2)

▼ ... Test Case 1
Optimal Assignment: [(1, 3), (2, 2), (3, 1)]
Total Cost: 16

Test Case 2
Optimal Assignment: [(1, 3), (2, 2), (3, 1)]
Total Cost: 17
```

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem. The program should:

```
Q Commands + Code + Text ▶ Run all ▼

[14]
✓ Os ▶ if value > best_value:
        best_value = value
        best_items = list(items)

    return best_items, best_value

weights1 = [2, 3, 1]
values1 = [4, 5, 3]
capacity1 = 4

weights2 = [1, 2, 3, 4]
values2 = [2, 4, 6, 3]
capacity2 = 6

items1, value1 = knapsack(weights1, values1, capacity1)
items2, value2 = knapsack(weights2, values2, capacity2)

print("Test Case 1")
print("Optimal Selection:", items1)
print("Total Value:", value1)

print("\nTest Case 2")
print("Optimal Selection:", items2)
print("Total Value:", value2)

▼ ... Test Case 1
Optimal Selection: [1, 2]
Total Value: 8

Test Case 2
Optimal Selection: [0, 1, 2]
Total Value: 12
```